

LAPORAN PRAKTIKUM STRUKTUR DATA

QUEUE

Disusun Oleh:

Digo Yuandra

NIM:2511533017

Dosen Pengampu:

Dr.Wahyudi S.T , M.T

Asisten Laboratorium:

Zahran Azaria Anvaya



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG 2026

KATA PENGANTAR

Dengan penuh rasa syukur, saya mengucapkan terima kasih kepada Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga laporan praktikum “Struktur Data Queue (Antrian)” ini dapat diselesaikan dengan baik.

Saya juga menyampaikan penghargaan dan terima kasih kepada dosen pengampu Mata Kuliah Praktikum Struktur Data serta para asisten laboratorium yang telah memberikan arahan, bimbingan, dan bantuan selama proses praktikum berlangsung.

Laporan ini disusun sebagai salah satu bentuk dokumentasi kegiatan praktikum yang bertujuan untuk memahami konsep struktur data Queue (antrian). Dalam laporan ini dibahas konsep dasar queue, prinsip kerja, serta penerapannya dalam studi kasus sederhana berupa sistem antrian loket, termasuk operasi dasar seperti enqueue dan dequeue, serta fitur tambahan seperti reverse data.

Saya menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat saya harapkan agar laporan ini dapat menjadi lebih baik di masa yang akan datang, serta dapat memberikan manfaat dan menambah pemahaman mengenai struktur data queue.

Padang, April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	
DAFTAR ISI.....	
BAB I PENDAHULUAN	
1.1 Latar Belakang	
1.2 Tujuan	
1.3 Manfaat.....	
BAB II PEMBAHASAN	
2.1 QueueArray_2511533017 & QueuearrayDriver_2511533017.....	
2.2 QueueLinkedList_2511533017.....	
2.3 ReverseData_2511533017	
2.4 IterasiQueue_2511533017	
2.5 Tugas	
BAB III KESIMPULAN	
3.1 Kesimpulan	
3.2 Saran.....	
DAFTAR PUSTAKA.....	

BAB I

PENDAHULUAN

1.1 Latar Belakang

Saat melakukan proses pengolahan data, ada kondisi tertentu yang tidak memungkinkan setiap elemen diakses secara bebas. Pada situasi seperti ini, diperlukan aturan khusus, yaitu data yang masuk lebih awal akan diproses terlebih dahulu. Aturan tersebut dikenal sebagai FIFO (*First In, First Out*) dan digunakan dalam struktur data Queue atau antrian.

Berbeda dengan Array atau ArrayList yang elemennya dapat diakses secara langsung melalui indeks, Queue memiliki mekanisme yang lebih teratur. Queue hanya melakukan operasi pada dua bagian utama, yaitu bagian depan (*front*) dan bagian belakang (*rear*). Oleh karena itu, struktur data ini banyak digunakan pada sistem antrian loket, pelayanan rumah sakit, maupun proses pengolahan data pada sistem komputer.

Melalui praktikum ini, pembahasan mengenai Queue bertujuan untuk memahami cara data dikelola secara berurutan, rapi, dan sistematis. Penerapan Queue dalam studi kasus sederhana juga membantu memperjelas alur kerja antrian serta operasi dasar yang digunakan, seperti *enqueue* dan *dequeue*.

1.2 Tujuan

1. Menjelaskan pengertian struktur data Queue beserta penerapan prinsip FIFO (*First In, First Out*).
2. Mengidentifikasi proses pengelolaan data pada Queue yang dilakukan secara berurutan.
3. Mengenal dan memahami fungsi operasi dasar Queue, seperti *enqueue* dan *dequeue*.
4. Menguraikan alur penggunaan Queue dalam contoh sistem antrian sederhana.
5. Melatih kemampuan berpikir logis dalam membuat program yang berkaitan dengan konsep antrian.

1.3 Manfaat

1. Memperdalam pemahaman mengenai penggunaan struktur data Queue dalam pemrograman.
2. Mengasah kemampuan berpikir secara logis dan sistematis dalam proses pengelolaan data.
3. Membantu memahami penerapan konsep antrian pada situasi sehari-hari.
4. Memberikan penjelasan umum mengenai mekanisme sistem yang bekerja dengan prinsip antrian.
5. Menambah pengalaman dalam mengkaji permasalahan yang berkaitan dengan struktur data.

BAB II

PEMBAHASAN

2.1 QueueArray_2511533017& QueuearrayDriver_2511533017

```
1 package Pekan4_2511533017;
2
3 public class QueueArray_2511533017 {
4     int front_3017, rear_3017, size_3017;
5     int capacity_3017;
6     int array_3017[];
7
8     public QueueArray_2511533017(int capacity_3017) {
9         this.capacity_3017 = capacity_3017;
10        front_3017 = this.size_3017 = 0;
11        this.rear_3017 = capacity_3017 - 1;
12        this.array_3017 = new int[this.capacity_3017];
13    }
14
15    boolean isFull(QueueArray_2511533017 queue) {
16        return (this.size_3017 == this.capacity_3017);
17    }
18
19    boolean isEmpty(QueueArray_2511533017 queue) {
20        return (this.size_3017 == 0);
21    }
22
23    void enqueue_3017(int item) {
24        if (isFull(this))
25            System.out.println("Queue Penuh!");
26        return;
27    }
28
29    int dequeue_3017() {
30        if (isEmpty(this))
31            return Integer.MIN_VALUE;
32
33        int item = this.array_3017[this.front_3017];
34        this.front_3017 = (this.front_3017 + 1) % this.capacity_3017;
35        this.size_3017 = this.size_3017 - 1;
36        return item;
37    }
38 }
```

```
28 }
29
30 int dequeue_3017() {
31     if (isEmpty(this))
32         return Integer.MIN_VALUE;
33
34     int item = this.array_3017[this.front_3017];
35     this.front_3017 = (this.front_3017 + 1) % this.capacity_3017;
36     this.size_3017 = this.size_3017 - 1;
37     return item;
38 }
39
40 int front_3017() {
41     if (isEmpty(this))
42         return Integer.MIN_VALUE;
43     return this.array_3017[this.front_3017];
44 }
45
46 int rear_3017() {
47     if (isEmpty(this))
48         return Integer.MIN_VALUE;
49     return this.array_3017[this.rear_3017];
50 }
51
52 // mencetak element antrian
53 void display() {
54     int i_3017;
55     if (isEmpty(this)) {
56         System.out.println("\nAntrian Kosong");
57         return;
58     }
59     //kunjungi dari belakang dan cetak
60     for (i_3017 = front_3017; i_3017 < rear_3017; i_3017++) {
61         System.out.printf("%d <- ", array_3017[i_3017]);
62     }
63     return;
64 }
```

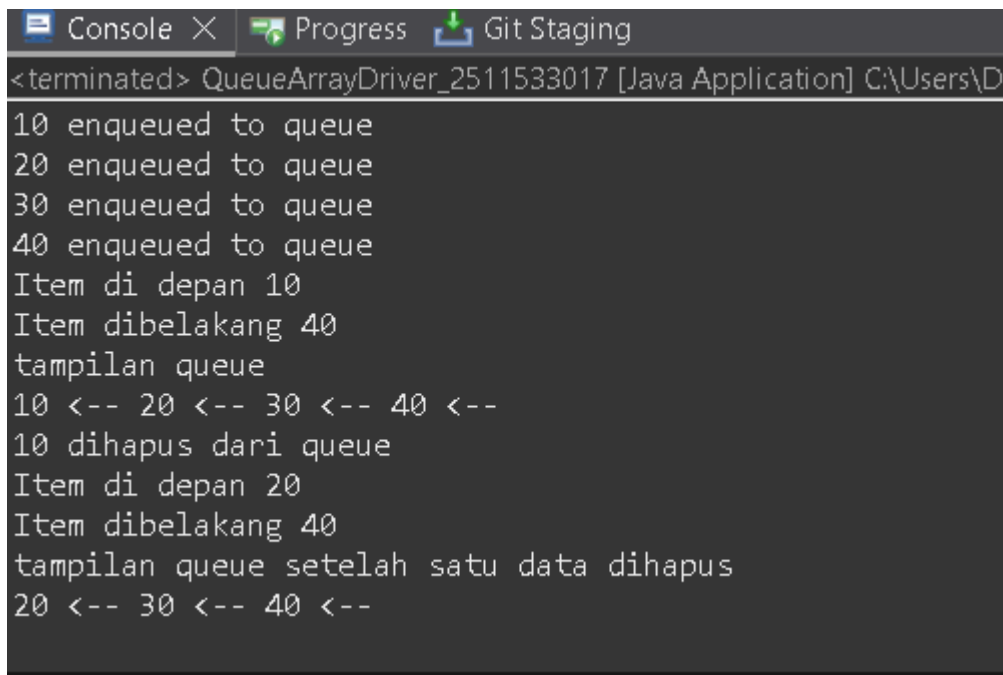
```
QueueArray_2511533017.java QueueArrayDriver_2511533017.java X
1 package Pekan4_2511533017;
2
3
4
5 public class QueueArrayDriver_2511533017 {
6
7     public static void main(String[] args) {
8         QueueArray_2511533017 queue_3017 = new QueueArray_2511533017 (1000) ;
9         queue_3017.enqueue_3017(10);
10        queue_3017.enqueue_3017(20);
11        queue_3017.enqueue_3017(30);
12        queue_3017.enqueue_3017(40);
13        System.out.println("Item di depan " + queue_3017.front_3017());
14        System.out.println("Item paling brlakang " + queue_3017.rear_3017());
15        System.out.println("tampilan queue");
16        queue_3017.display();
17        System.out.println();
18        System.out.println(queue_3017.dequeue_3017() + " dihapus dari queue");
19        System.out.println("item di depan:" + queue_3017.front_3017());
20        System.out.println("item di belakang: " + queue_3017.rear_3017());
21        System.out.println("tampilan queue setelah satu data dihapus");
22        queue_3017.display();
23    }
24 }
25
26 }
27
```

Pembahasan:

1. Pada program driver, dibuat objek queue_3017 dari class QueueArray_2511533017 dengan kapasitas 1000. Pada kondisi awal, queue masih kosong karena nilai size = 0.
2. Perintah enqueue_3017(10), enqueue_3017(20), enqueue_3017(30), dan enqueue_3017(40) digunakan untuk memasukkan data ke dalam queue secara berurutan.
3. Setiap kali method enqueue_3017() dijalankan, program terlebih dahulu memeriksa apakah queue sudah penuh melalui method isFull_3017(). Jika queue belum penuh, posisi rear akan digeser menggunakan rumus $(rear + 1) \% capacity$, kemudian data dimasukkan ke dalam array dan nilai size bertambah.
4. Setelah data 10, 20, 30, dan 40 berhasil dimasukkan, susunan queue menjadi 10 sebagai elemen paling depan, kemudian 20, 30, dan 40 sebagai elemen paling belakang.
5. Perintah System.out.println("Item di depan " + queue_3017.front_3017()); digunakan untuk menampilkan data yang berada pada posisi paling depan queue.
6. Perintah System.out.println("Item dibelakang " + queue_3017.rear_3017()); digunakan untuk menampilkan data yang berada pada posisi paling belakang queue.
7. Method queue_3017.display_3017(); berfungsi untuk menampilkan seluruh isi queue mulai dari elemen depan hingga elemen belakang.
8. Method queue_3017.dequeue_3017(); digunakan untuk mengambil sekaligus menghapus elemen yang berada pada posisi paling depan queue.
9. Sebelum proses penghapusan dilakukan, program memeriksa kondisi queue melalui method isEmpty_3017() untuk memastikan bahwa queue tidak dalam keadaan kosong.
10. Data pertama, yaitu 10, diambil dari posisi front. Setelah itu, posisi front digeser dan nilai size dikurangi, sehingga data 10 keluar dari queue.
11. Setelah proses dequeue dilakukan, susunan queue berubah menjadi 20 sebagai elemen paling depan, diikuti oleh 30, dan 40 sebagai elemen paling belakang.

12. Pemanggilan kembali method `front_3017()`, `rear_3017()`, dan `display_3017()` bertujuan untuk menampilkan kondisi queue setelah satu elemen dihapus.
13. Berdasarkan hasil pengujian pada program driver, terlihat bahwa data yang pertama keluar adalah 10, yaitu data yang pertama kali dimasukkan ke dalam queue.
14. Hal tersebut menunjukkan bahwa struktur data queue bekerja sesuai dengan prinsip FIFO (*First In, First Out*), yaitu data yang masuk lebih awal akan diproses atau dikeluarkan lebih dahulu.

Output:



```
<terminated> QueueArrayDriver_2511533017 [Java Application] C:\Users\D
10 enqueued to queue
20 enqueued to queue
30 enqueued to queue
40 enqueued to queue
Item di depan 10
Item dibelakang 40
tampilan queue
10 <-- 20 <-- 30 <-- 40 <--
10 dihapus dari queue
Item di depan 20
Item dibelakang 40
tampilan queue setelah satu data dihapus
20 <-- 30 <-- 40 <--
```

2.2 QueueLinkedList_2511533017

```
1 package Pekan4_2511533017;
2
3 import java.util.*;
4
5
6 public class QueueLinkedList_2511533017 {
7
8     public static void main(String[] args) {
9         Queue<Integer> q_3017 = new LinkedList<>();
10
11         // tambah elemen {0, 1, 2, 3, 4, 5} ke antrian
12         for (int i_3017 = 0; i_3017 < 6; i_3017++) {
13             q_3017.add(i_3017);
14         }
15
16         // Menampilkan isi antrian.
17         System.out.println("Elemen Antrian " + q_3017);
18
19         // Untuk menghapus kepala antrian.
20         int hapus_3017 = q_3017.remove();
21         System.out.println("Hapus elemen = " + hapus_3017);
22         System.out.println(q_3017);
23
24         // Untuk melihat antrian terdepan
25         int depan_3017 = q_3017.peek();
26         System.out.println("Kepala Antrian = " + depan_3017);
27
28         int banyak_3017 = q_3017.size();
29         System.out.println("Size Antrian = " + banyak_3017);
30     }
31 }
```

Pembahasan:

1. Perintah `Queue<Integer> q_3017 = new LinkedList<>();` digunakan untuk membuat queue dengan bantuan `LinkedList`, sehingga pengelolaan antrian dapat dilakukan secara otomatis tanpa membuat struktur queue secara manual.
2. Perulangan `for (int i = 0; i < 6; i++) q_3017.add(i);` berfungsi untuk memasukkan data dari 0 sampai 5 ke dalam queue secara berurutan.
3. Method `add()` akan menempatkan setiap data baru pada bagian belakang antrian.
4. Setelah seluruh data dimasukkan, isi queue tersusun menjadi 0, 1, 2, 3, 4, dan 5.
5. Perintah `System.out.println(q_3017);` digunakan untuk menampilkan seluruh isi queue.
6. Method `q_3017.remove();` berfungsi untuk menghapus elemen yang berada pada bagian paling depan, yaitu 0.
7. Setelah elemen 0 dihapus, isi queue berubah menjadi 1, 2, 3, 4, dan 5.

8. Method `q_3017.peek()`; digunakan untuk melihat elemen paling depan tanpa menghapusnya dari queue. Pada kondisi tersebut, elemen yang tampil adalah 1.
9. Method `q_3017.size()`; digunakan untuk mengetahui jumlah elemen yang masih ada di dalam queue, yaitu 5 elemen.
10. Berdasarkan hasil tersebut, dapat disimpulkan bahwa queue bekerja sesuai dengan prinsip FIFO (*First In, First Out*), yaitu data yang pertama masuk akan menjadi data pertama yang keluar.

Output:

```
Console X Progress Git Staging
<terminated> QueueLinkedList_2511533017 [Java A
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus elemen = 0
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antrian = 5
```

2.3 ReverseData_2511533017

```
QueueArray_2511533017.java QueueArrayDriver_2511533017.java QueueLinkedList_2511533017.java
1 package Pekan4_2511533017;
2 import java.util.LinkedList;
3
4
5
6 public class ReverseData_2511533017 {
7
8
9     public static void main(String[] args) {
10         Queue<Integer> q_3017 = new LinkedList<Integer>();
11         q_3017.add(1);
12         q_3017.add(2);
13         q_3017.add(3);
14         System.out.println("sebelum reverse" + q_3017);
15         Stack<Integer> s = new Stack<Integer>();
16         while (!q_3017.isEmpty()) { // Q -> S
17             s.push(q_3017.remove());
18
19         }
20         while (!s.isEmpty()) { // S -> Q
21             q_3017.add(s.pop());
22
23         }
24         System.out.println("sesudah reverse= " + q_3017); // [3, 2, 1]
25
26
27
28     }
29
30 }
31
```

1. Perintah `Queue<Integer> q_3017 = new LinkedList<>();` digunakan untuk membuat queue dengan `LinkedList` sebagai tempat penyimpanan datanya.
2. Perintah `q_3017.add(1)`, `q_3017.add(2)`, dan `q_3017.add(3)` berfungsi untuk memasukkan data ke dalam queue secara berurutan, sehingga isi awal queue adalah 1, 2, dan 3.
3. Perintah `System.out.println("Sebelum reverse" + q_3017);` digunakan untuk menampilkan isi queue sebelum proses pembalikan data dilakukan.
4. Perintah `Stack<Integer> s_3017 = new Stack<>();` digunakan untuk membuat stack sebagai struktur data bantuan dalam proses *reverse*.
5. Pada perulangan `while (!q_3017.isEmpty())`, seluruh elemen di dalam queue dipindahkan ke stack menggunakan method `remove()` dan `push()`.
6. Setelah elemen dipindahkan ke stack, urutannya menjadi terbalik karena stack bekerja dengan prinsip LIFO (*Last In, First Out*).
7. Pada perulangan `while (!s_3017.isEmpty())`, elemen dari stack dikembalikan lagi ke queue menggunakan method `pop()` dan `add()`.
8. Perintah `System.out.println("Sesudah reverse= " + q_3017);` digunakan untuk menampilkan hasil akhir queue setelah dibalik, yaitu menjadi 3, 2, dan 1.

Output:

```
<terminated> ReverseData_25115330
sebelum reverse[1, 2, 3]
sesudah reverse= [3, 2, 1]
```

2.4 IterasiQueue_2511533017

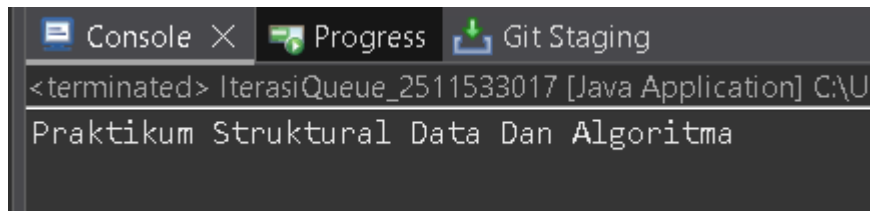
```
QueueArray_2511533017.java QueueArrayDriver_2511533017.java IterasiQu

1 package Pekan4_2511533017;
2 import java.util.*;
3 public class IterasiQueue_2511533017 {
4
5     public static void main(String args[]) {
6         Queue<String> q_3017 = new LinkedList<>();
7
8         q_3017.add("Praktikum");
9         q_3017.add("Struktural");
10        q_3017.add("Data");
11        q_3017.add("Dan");
12        q_3017.add("Algoritma");
13        Iterator<String> iterator_3017 = q_3017.iterator();
14        while (iterator_3017.hasNext()) {
15            System.out.print(iterator_3017.next() + " ");
16        }
17
18
19
20    }
21
22 }
23
```

Pembahasan:

1. Perintah `Queue<String> q_3017 = new LinkedList<>();` digunakan untuk membuat queue dengan tipe data String menggunakan bantuan `LinkedList`.
2. Method `q_3017.add(...)` berfungsi untuk memasukkan beberapa data ke dalam queue, yaitu "Praktikum", "Struktur", "Data", "Dan", dan "Algoritma".
3. Setelah seluruh data dimasukkan, urutan isi queue menjadi Praktikum, Struktur, Data, Dan, dan Algoritma.
4. Perintah `Iterator<String> iterator_3017 = q_3017.iterator();` digunakan untuk membuat iterator yang berfungsi menelusuri setiap elemen di dalam queue.
5. Perulangan `while (iterator_3017.hasNext())` dijalankan selama masih terdapat elemen berikutnya yang dapat dibaca.
6. Method `iterator_3017.next()` digunakan untuk mengambil elemen queue satu per satu secara berurutan dari bagian depan sampai bagian belakang.
7. Perintah `System.out.print(...)` berfungsi untuk menampilkan setiap elemen queue pada baris yang sama.
8. Melalui proses iterasi tersebut, semua elemen di dalam queue dapat ditampilkan tanpa menghapus atau mengubah data yang tersimpan.

Ouput:



```
<terminated> IterasiQueue_2511533017 [Java Application] C:\U
Praktikum Struktural Data Dan Algoritma
```

2.5 Tugas

1. Queue_2511533017

```
1 package Pekan4_2511533017;
2
3 public class Queue_2511533017 {
4     int front_3017, rear_3017, max_3017;
5     String queue_3017[];
6
7     public Queue_2511533017(int max_3017) {
8         this.max_3017 = max_3017;
9         queue_3017 = new String[max_3017];
10        front_3017 = -1;
11        rear_3017 = -1;
12    }
13
14    public boolean isEmpty_3017() {
15        return (front_3017 == -1);
16    }
17
18    public boolean isFull_3017() {
19        return (rear_3017 == max_3017 - 1);
20    }
21
22    public void enqueue_3017(String data_3017) {
23        if (isFull_3017()) {
24            System.out.println("Antrian penuh");
25        } else {
26            if (isEmpty_3017()) {
27                front_3017 = 0;
28            }
29            rear_3017++;
30            queue_3017[rear_3017] = data_3017;
31            System.out.println("Data berhasil ditambahkan ke antrian");
32        }
33    }
34
35    public void dequeue_3017() {
36        if (isEmpty_3017()) {
37            System.out.println("Antrian kosong");
38        } else {
```

```
49     if (isEmpty_3017()) {
50         System.out.println("Antrian kosong");
51     } else {
52         System.out.println("Isi antrian:");
53         int no= 1;
54         for (int i = front_3017; i <= rear_3017; i++) {
55             System.out.println(no + ". " + queue_3017[i]);
56             no++;
57         }
58     }
59 }
60
61 public void reverse_3017() {
62     if (isEmpty_3017()) {
63         System.out.println("Antrian kosong!");
64     } else {
65         int start = front_3017;
66         int end = rear_3017;
67         while (start < end) {
68             String temp = queue_3017[start];
69             queue_3017[start] = queue_3017[end];
70             queue_3017[end] = temp;
71             start++;
72             end--;
73         }
74     }
75 }
76 }
```

Pembahasan:

1. Variabel `front_3017`, `rear_3017`, `max_3017`, dan array `queue_3017[]` digunakan untuk menyimpan informasi posisi antrian, kapasitas maksimum, serta data yang ada di dalam queue.
2. Constructor `Queue_2511533017(int max_3017)` berfungsi untuk mengatur kapasitas maksimum antrian saat objek queue dibuat.
3. Perintah `queue_3017 = new String[max_3017]`; digunakan untuk membuat array sesuai dengan ukuran kapasitas yang telah ditentukan.
4. Nilai awal `front_3017 = -1` dan `rear_3017 = -1` menunjukkan bahwa queue masih dalam keadaan kosong.
5. Method `isEmpty_3017()` digunakan untuk memeriksa apakah antrian kosong dengan melihat apakah nilai `front_3017` masih -1.
6. Method `isFull_3017()` berfungsi untuk mengecek apakah antrian sudah penuh, yaitu ketika posisi `rear_3017` sudah mencapai `max_3017 - 1`.
7. Method `enqueue_3017(String data_3017)` digunakan untuk menambahkan data baru ke dalam antrian.
8. Jika antrian masih kosong, maka posisi `front_3017` akan diubah menjadi 0. Setelah itu, nilai `rear_3017` bertambah dan data dimasukkan ke dalam array.
9. Method `dequeue_3017()` digunakan untuk menghapus data yang berada pada bagian depan antrian.
10. Jika antrian hanya memiliki satu data, maka nilai `front_3017` dan `rear_3017` dikembalikan menjadi -1. Namun, jika masih terdapat data lain, maka posisi `front_3017` akan digeser ke elemen berikutnya.
11. Method `display_3017()` berfungsi untuk menampilkan seluruh isi antrian mulai dari posisi depan hingga posisi belakang.
12. Method `reverse_3017()` digunakan untuk membalik urutan data di dalam queue dengan cara menukar elemen dari bagian depan dan bagian belakang.

2. AntrianLoket_2511533017

```
1 package Pekan4_2511533017;
2 import java.util.*;
3
4 public class AntrianLoket_2511533017 {
5
6     public static void main(String[] args) {
7         Scanner input = new Scanner(System.in);
8         Queue_2511533017 antrian = new Queue_2511533017(10);
9
10        int pilihan;
11        do {
12            System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
13            System.out.println("1. Tambah Antrian");
14            System.out.println("2. Hapus Antrian");
15            System.out.println("3. Tampilkan Antrian");
16            System.out.println("4. Reverse");
17            System.out.println("5. Keluar");
18            System.out.print("Pilih menu: ");
19            pilihan = input.nextInt();
20            input.nextLine();
21
22            switch (pilihan) {
23                case 1:
24                    System.out.print("Masukkan nama pelanggan: ");
25                    String nama = input.nextLine();
26                    antrian.enqueue_3017(nama);
27                    break;
28
29                case 2:
30                    antrian.dequeue_3017();
31                    break;
32
33                case 3:
34                    antrian.display_3017();
35                    break;
36
37                case 4:
38                    antrian.reverse_3017();
39
40                case 5:
41                    System.out.println("Program selesai");
42                    break;
43
44                default:
45                    System.out.println("Pilihan tidak valid");
46            }
47        } while (pilihan != 5);
48
49        input.close();
50    }
51 }
52
53 }
```


Pembahasan:

`Queue_2511533017 antrian = new Queue_2511533017(10);` digunakan untuk membuat objek antrian dengan kapasitas maksimal 10 data sebagai tempat menyimpan nama pelanggan.

`int pilihan;` berfungsi sebagai variabel untuk menampung pilihan menu yang dimasukkan oleh pengguna.

Perulangan `do { ... } while (pilihan != 5);` digunakan agar program tetap berjalan dan terus menampilkan menu sampai pengguna memilih menu keluar.

Perintah `System.out.println("=== PROGRAM ANTRIAN LOKET ===");` digunakan untuk menampilkan menu program antrian loket kepada pengguna.

`pilihan = input.nextInt();` berfungsi untuk membaca input pilihan yang dimasukkan oleh pengguna.

Struktur `switch (pilihan)` digunakan untuk menjalankan perintah tertentu sesuai dengan menu yang dipilih.

Pada case 1, program digunakan untuk menambahkan data pelanggan ke dalam antrian.

Perintah `antrian.enqueue_3017(nama);` berfungsi memasukkan nama pelanggan ke bagian belakang antrian.

Pada case 2, program digunakan untuk menghapus data pelanggan dari antrian. Perintah `antrian.dequeue_3017();` akan menghapus data yang berada di bagian paling depan sesuai dengan prinsip FIFO.

case 3 digunakan untuk menampilkan isi antrian yang sedang tersimpan di dalam program. Perintah `antrian.display_3017()`; berfungsi untuk menampilkan seluruh data pelanggan yang ada di dalam antrian.

case 4 digunakan untuk membalik urutan data pada antrian. Perintah `antrian.reverse_3017()`; akan membalik susunan data dalam antrian, kemudian hasilnya dapat ditampilkan.

case 5 digunakan untuk menghentikan program. Perintah `System.out.println("Program selesai");` berfungsi untuk menampilkan pesan bahwa program telah selesai dijalankan.

Bagian default digunakan untuk menangani pilihan yang tidak sesuai dengan menu yang tersedia. Perintah `System.out.println("Pilihan tidak valid!");` akan menampilkan pesan kesalahan apabila pengguna memasukkan pilihan yang salah.

Output:

```
== PROGRAM ANTRIAN LOKET ==  
. Tambah Antrian  
. Hapus Antrian  
. Tampilkan Antrian  
. Reverse  
. Keluar  
Pilih menu: 1  
Masukkan nama pelanggan: digo  
Data berhasil ditambahkan ke antrian  
  
== PROGRAM ANTRIAN LOKET ==  
. Tambah Antrian  
. Hapus Antrian  
. Tampilkan Antrian  
. Reverse  
. Keluar  
Pilih menu: 2  
digo telah dilayani  
  
== PROGRAM ANTRIAN LOKET ==
```

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum mengenai Queue, dapat disimpulkan bahwa Queue merupakan struktur data linier yang menerapkan prinsip FIFO (*First In, First Out*), yaitu data yang pertama kali masuk akan menjadi data pertama yang keluar. Queue dapat direpresentasikan dalam beberapa bentuk, seperti menggunakan array maupun `LinkedList`.

Operasi dasar pada Queue, seperti *enqueue*, *dequeue*, dan pengecekan kondisi antrian, memiliki waktu eksekusi yang cepat, yaitu $O(1)$, sehingga cukup efisien dalam proses pengelolaan data secara berurutan. Penerapan Queue dalam praktikum ini, seperti pada sistem antrian loket, menunjukkan bahwa konsep Queue banyak digunakan dalam berbagai situasi yang membutuhkan pengolahan data secara teratur dan terstruktur. Selain itu, Queue juga dapat dikembangkan dengan fitur tambahan, salah satunya adalah proses *reverse data*.

Dengan demikian, Queue tidak hanya penting untuk dipahami secara teori, tetapi juga memiliki manfaat praktis dalam membantu menyelesaikan permasalahan nyata yang berkaitan dengan pengelolaan data secara sistematis.

3.2 Saran

Sebaiknya sebelum praktikum dilaksanakan, dosen menyelenggarakan sesi pra-praktikum di kelas sebagai pengantar materi. Hal ini bertujuan agar mahasiswa memiliki pemahaman awal yang lebih baik mengenai materi yang akan dipraktikkan.

Selain itu, materi praktikum sebaiknya dibagikan terlebih dahulu melalui platform iLearn, sehingga mahasiswa memiliki waktu yang cukup untuk mempelajari, memahami, dan mempersiapkan diri sebelum kegiatan praktikum berlangsung. Dengan persiapan tersebut, potensi kepanikan, kebingungan, maupun kesalahan selama praktikum dapat diminimalkan.

DAFTAR PUSTAKA

- [1] Oracle, “Queue Interface (Java Platform SE Documentation),” [Online]. Available: <https://docs.oracle.com/javase/8/docs/api/java/util/Queue.html>

